Concept Types

Concept Types

Vaultで使用する概念の型。

Entity

存在するもの。

例
Human
Organization
Institution
Technology

Process

変化や行動。

例
Decision
Learning
Coordination

Structure

仕組みや構造。

例
Hierarchy
Network
Market

State

状態。

例
Stability
Crisis
Collapse

Relation

概念同士の関係。

例
causes
part_of
explains

Domain Types

Knowledge Layers

Knowledge Layers

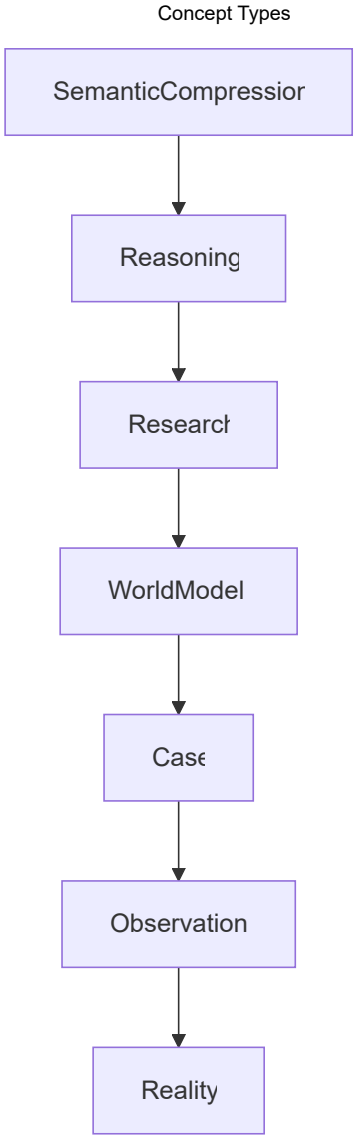
Knowledge Layers は Vault における知識の機能階層を定義する。

Vault は単なる知識管理ではなく、

- Knowledge OS
- Reasoning OS
- Research OS

の三層構造で動作する。

Layer Structure



Layer Overview

Layer	役割
Semantic Compression	LLM理解の高速化
Reasoning	思考制御
Research	理論検証
World Model	世界の説明
Case	事例
Observation	観測

Layer	役割
Reality	現実

---

1 Semantic Compression Layer

Vaultの知識をAIが理解しやすい形に圧縮する。

主なノート

- Kernel Map
- Theory Map
- Model Map
- Structure Map
- Pattern Map
- Reasoning Map

目的

- コンテキスト節約
- 推論高速化

---

2 Reasoning Layer

AIと人間の思考手順を制御する。

主なノート

- Problem Classifier
- Model Selector
- Reasoning Pipeline
- Evaluation Rules
- Output Structure

---

3 Research Layer

理論を検証し更新する層。

主なノート

- Research Program
- Hypothesis
- Test
- Dataset
- Result

---

4 World Model Layer

世界の仕組みを説明する。

主なノート

- Kernel
- Theory
- Model
- Structure
- Mechanism
- Pattern

---

5 Case Layer

理論の適用対象となる具体事例。

例

- 歴史事件
- 組織事例
- 社会現象

---

6 Observation Layer

観測された事実。

例

- 発言
- 行動
- 数値

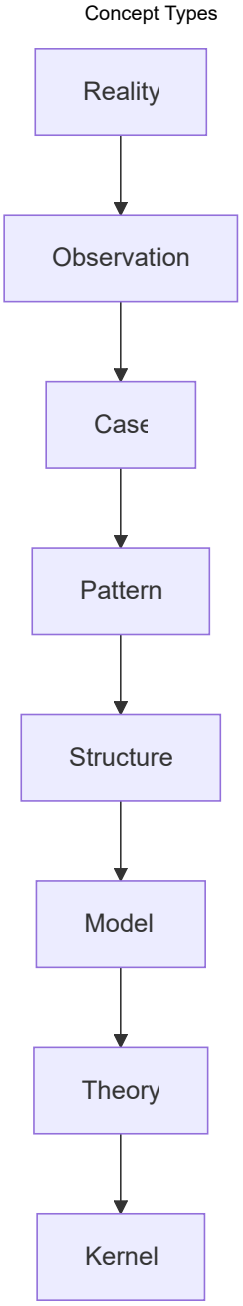
- 史料

---

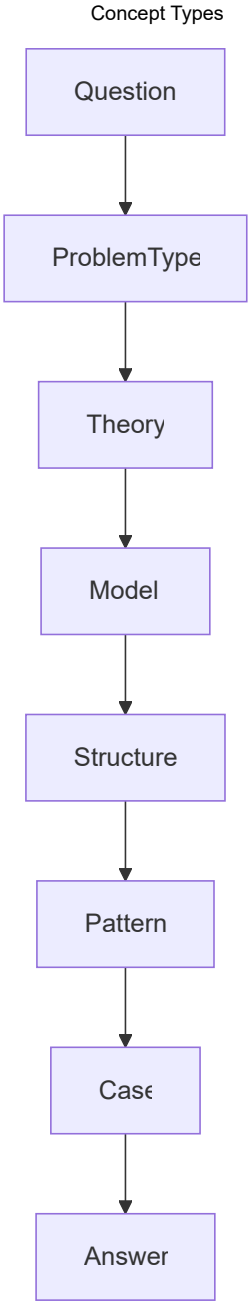
7 Reality Layer

Vault外部の現実世界。

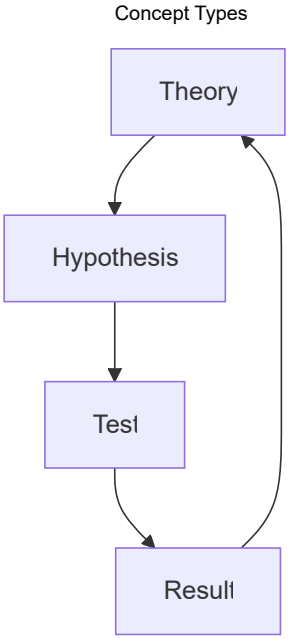
Knowledge Flow



Reasoning Flow



Research Flow



Layer Interaction

Knowledge Layers は次の3つのOSを形成する。

```
Knowledge OS
Reasoning OS
Research OS
```

関連ノート

[02_zettelkasten/01_ontology/Ontology]
[99_oldzettelkasten/Concept Hub]
[Semantic Compression]
[Inference Hub]

Ontology

Ontology

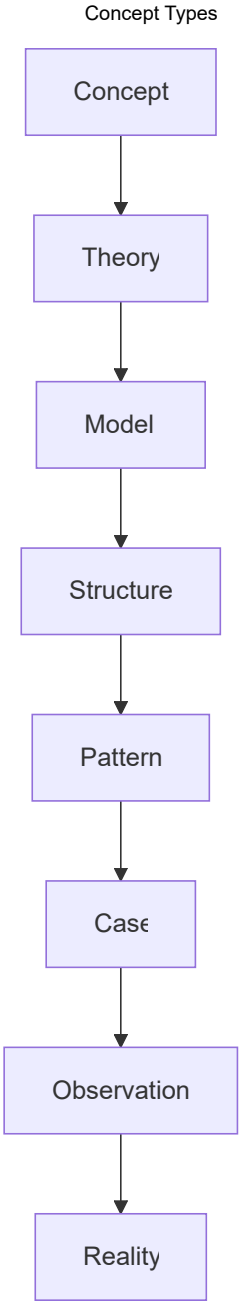
Ontology は Vault における知識の存在論を定義する。
すべてのノートは Ontology に定義された型に属する。

Ontology は

- Concept Graph
- Theory
- Model
- Structure
- Pattern
- Case

の関係を統一する。

Knowledge Ontology



Layer Definitions

Concept

世界を記述する基本語彙。

例

- Human
- Power
- Institution
- Decision

Concept は Vault の語彙体系である。

Theory

複数の Concept 関係を説明する一般仮説。

例

- Limited Rationality Theory
- Institutional Theory
- Power Concentration Theory

Model

Theory を形式化した分析モデル。

例

- Expected Value Model
- Bureaucracy Model
- Network Model

Structure

関係配置またはシステム構造。

例

- Hierarchy
- Network
- Market
- Bureaucracy

Pattern

繰り返し観測される現象。

例

- Conformity
- Bureaucratic delay
- Power concentration

Case

具体事例。

例

- French Revolution
- Microsoft
- Tokugawa Shogunate

Observation

観測された事実。

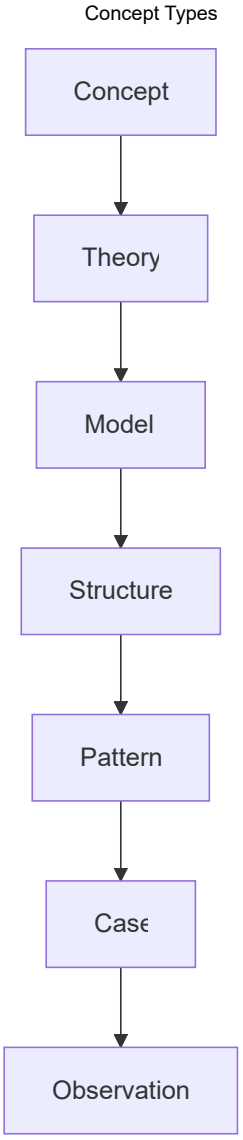
例

- 発言
- 行動
- 数値
- 記録

Reality

Vault 外部の現実世界。

Ontology Relations



Layer Roles

Layer	Role

Concept | 語彙 |
Theory | 説明 |
Model | 形式化 |
Structure | メカニズム |
Pattern | 再現現象 |
Case | 実例 |

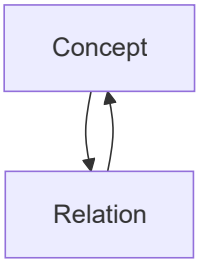
Observation | 記録 |
Reality | 現実 |

Ontology Design Principles

1. 上位層ほど抽象度が高い
2. 下位層ほど具体的
3. Theory は Model を含む
4. Pattern は Structure の現れ
5. Case は Pattern の実例

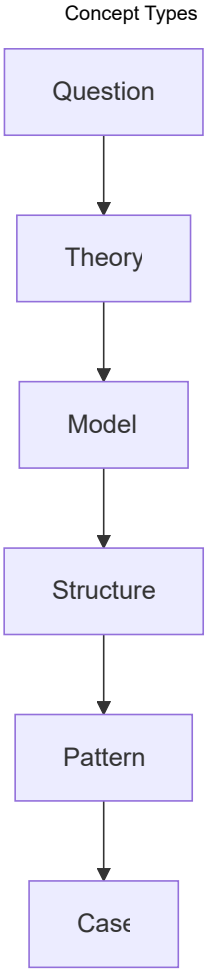
Relation to Concept Graph

Concept Graph は Ontology の上で構築される。



Relation to Reasoning

推論は Ontology を上から下へ辿る。



関連ノート

99_oldzettelkasten/Concept Hub

02_zettelkasten/01_ontology/Concept Types

99_oldzettelkasten/04_knowledge_graph/Relation Types

Causal Relations

Concept Relation Rule

Concept Relation Rule

このノートは Vault 内の概念関係（Concept Relation）の使用ルールを定義する。

Vaultは知識グラフであり、ノートは概念ノードである。
関係（relation）はノード間の意味的リンクである。

関係は自由に作るのではなく **layerごとに使用可能な関係を制限する**。

---

1 使用可能な関係

Vaultで使用する基本関係は次の通り。

relation	意味
is_a	上位概念
part_of	構造の一部
derived_from	派生元
instance_of	具体例
causes	因果
caused_by	原因
explains	説明
related	横関係

---

2 layer構造

Vaultの基本階層は次の通り。

kernel
↓
structure
↓
mechanism
↓
pattern

↓
case

補助レイヤー

- model
- domain
- method
- concept

---

3 layerごとの関係ルール

Kernel

世界の基本原理。

使用可能 relation

- explains
- related

例

- 社会性原理 ：explains → 同調
- Kernel同士は related

のみ。

Structure

関係構造。

使用可能 relation

- is_a
- part_of
- derived_from
- related
- 例
- 寡占構造
- is_a → 市場構造

Mechanism

因果メカニズム。

使用可能 relation

- derived_from
- causes
- caused_by
- related

例
同調

- derived_from → 社会性原理
- causes → 集団行動

Pattern

繰り返し発生する現象。

使用可能 relation

- derived_from
- explains
- related

例：炎上

- derived_from → 同調

Case

具体事例。

使用可能 relation

- instance_of
- related

例：ランス革命

- instance_of → 革命パターン

Model

抽象モデル。

使用可能 relation

- explains
- derived_from
- related

例
期待価値モデル

- explains → 意思決定

Domain

学問分野。

使用可能 relation：

- part_of
- related

例

- 都市計画
- part_of → 社会科学

---

4 sibling関係

兄弟関係は、is_a の共有で判断する。

例

- 同調
- 社会的証明
- 権威影響

これらは、is_a → 社会心理メカニズムを共有するため siblings である。

---

5 構造リンク原則

Vaultでは次のリンクを推奨する。

- Case → Pattern
- Pattern → Mechanism
- Mechanism → Structure
- Structure → Kernel

このリンクがあると知識グラフが安定する。

---

6 禁止事項

次の関係は禁止。

- Case → Kernel
- Pattern → Case
- Kernel → Case

理由 -
抽象階層が崩壊するため。

---

7 Graph構造

Vaultの知識グラフは次の構造を持つ。

Kernel
↓
Structure
↓
Mechanism
↓
Pattern
↓
Case

横方向

- related

---

8 Vault QA

QAスクリプトは次を検査する。

- relation誤用
- layer逆転
- 孤立ノート

---

9 設計原則

Concept relation は、少なく明確にする。

理想 relation 数：5〜8

---

10 まとめ

Vaultの概念関係は

- 階層関係
- 因果関係
- 横関係

の3種類に整理する。
これにより Vault は **知識グラフとして安定する**。

Property Taxonomy

Relation Ontology

Vaultのノートは単なるリンクではなく
意味付き関係（Relation）で接続する。

これによりVaultは

Knowledge Graph
として機能する。

Relation Types

Vaultで使用するRelationは以下の10種類。

relation	意味

is_a | 上位概念 |
part_of | 構成要素 |
instance_of | 具体例 |
derived_from | 派生 |
causes | 原因 |
caused_by | 原因 |
explains | 説明 |
analogous_to | 類似 |
contradicts | 矛盾 |
related_to | 一般関連 |

---

1 is_a

上位概念との関係。

例

社会的証明
is_a
社会心理メカニズム

---

2 part_of

構造の構成要素。

例

注意資源制約
part_of
限定合理性

---

3 instance_of

具体例。

例

エムス電報事件
instance_of
引き金事件

---

4 derived_from

派生概念。

例

プロスペクト理論
derived_from
期待効用理論

---

5 causes

原因。

例

情報非対称
causes
逆選択

---

6 caused_by

原因の逆。

例

逆選択
caused_by
情報非対称

---

7 explains

説明関係。

例

プロスペクト理論
explains
リスク回避行動

---

8 analogous_to

類似。

例

都市交通ネットワーク
analogous_to
血管ネットワーク

---

9 contradicts

矛盾。

例

合理的選択理論
contradicts
プロスペクト理論

---

10 related_to

弱い関連。

例

写真フィールドワーク
related_to
都市観察

Relation Usage Rule

原則

強い関係を優先する。

順序

1
is_a

2
part_of

3
instance_of

4
derived_from

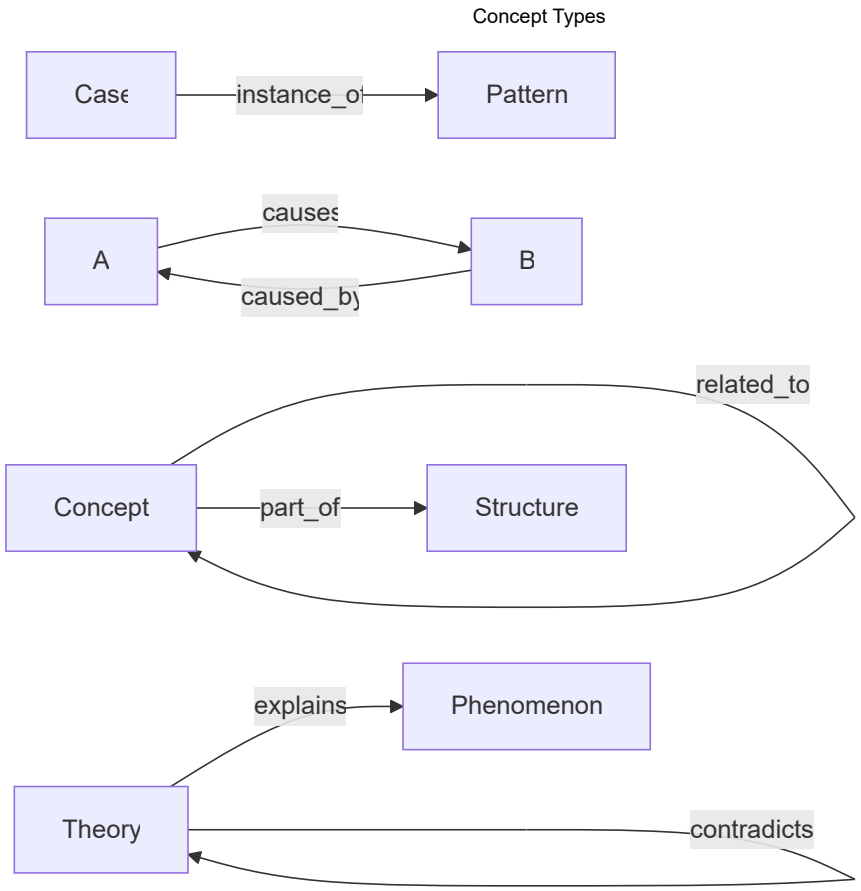
5
causal

6
explanatory

7
analogy

8
weak relation

Relation Graph



Design Philosophy

Vaultは

知識のリストではない。

知識グラフである。

```
concept
↓
mechanism
↓
pattern
↓
case
```

すべてのノードは
Relationで接続される。

Edge Type

Edge Type

目的

関係の種類を統一

種類

- is-a
- part-of
- causes
- similar
- contrast

ルール

- 必ずラベルを付ける
- 無名リンク禁止

参照

- 02_zettelkasten/01_ontology/Relation Types

Naming Rule

Naming Rule

目的

ノート名の一貫性を保つ

ルール

1. 日本語 + 機能

例:

- 情報拡散メカニズム
- 放射型構造

---

2. レイヤーを含める

- ○○構造
- ○○メカニズム
- ○○パターン

---

3. 曖昧語禁止

- 「仕組み」「やつ」などNG

---

4. 重複禁止

似た名前は統合

人力チェック

- 一目で意味が分かるか？
- 他と被ってないか？

関連

- 02_zettelkasten/01_ontology/Ontology
- 02_zettelkasten/01_ontology/Concept Types

Node Type

Node Type

目的

ノードの種類を明確化

種類

- Kernel Node
- Concept Node
- Structure Node
- Mechanism Node
- Pattern Node
- Case Node

判断方法

→ Knowledge Layers参照

ルール

1ノード1タイプ

Problem Type

Problem Type

問題は構造によって種類が異なる。
Problem Type は**問題の性質を分類する枠組み**である。

問題の種類を誤ると
解決方法も誤る。

問題の基本分類

```
Observation Problem
Understanding Problem
Explanation Problem
Prediction Problem
Decision Problem
Design Problem
```

---

1 Observation Problem

現象を把握する問題。

例

- 何が起きているのか
- 規模はどの程度か
- どこで発生しているか

例

- 交通渋滞の発生地点
- 人口減少の地域分布

使用ツール

- observation
- Measurement（Measurement）
- Data Collection

2 Understanding Problem

構造を理解する問題。

例

- この街の都市構造はどうなっているか
- この産業の構造はどうなっているか

使用ツール

- Structure Analysis
- System Mapping

---

3 Explanation Problem

原因を説明する問題。

例

- なぜ交通渋滞が起きるのか
- なぜ国家が崩壊するのか

使用ツール

- Causal Reasoning
- Mechanism Identification

---

4 Prediction Problem

未来を予測する問題。

例

- 人口はどう変化するか
- 技術はどう発展するか

使用ツール

- Trend Analysis
- Scenario Analysis

5 Decision Problem

複数の選択肢から選ぶ問題。

例

- どの戦略を取るべきか
- どの政策を採用するか

使用ツール

- Decision Analysis
- Expected Value Model

---

6 Design Problem

新しい解決策を設計する問題。

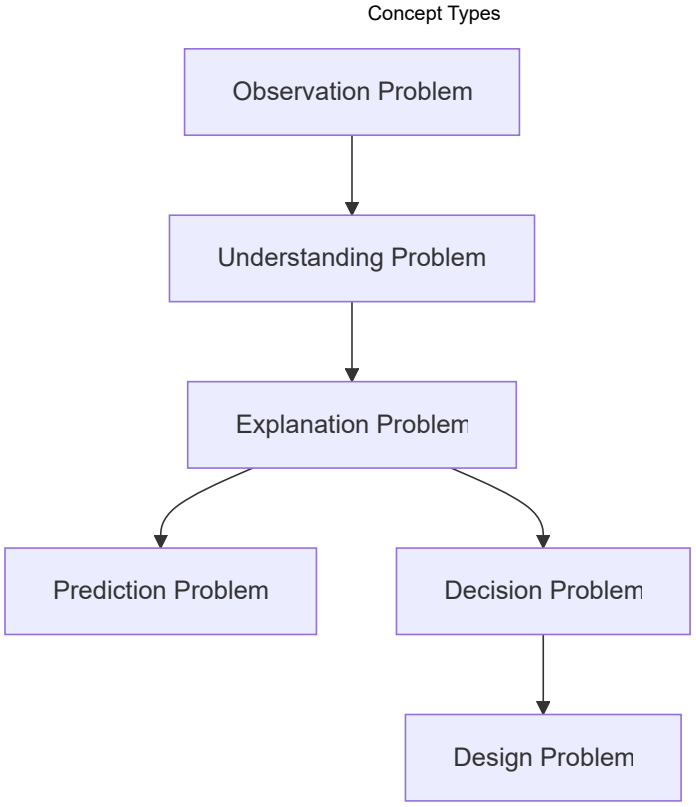
例

- 新しい制度を設計する
- 新しいビジネスモデルを作る

使用ツール

- Solution Design
- System Design

Problem Type Map



Thinking Engineとの関係

```
Problem
   ↓
Problem Type
   ↓
Diagnostic Questions
   ↓
Reasoning
   ↓
Decision
   ↓
Solution
```

関連ノート

- Diagnostic Questions

- Research Loop
- 02_zettelkasten/03_thinking_engine/Thinking Engine
- Hypothesis

Relation Types

Relation Types

目的

ノード間の関係を定義する

基本関係

1. is-a（分類）

例: 都市 is-a 空間

---

2. part-of（構成）

例: 駅 part-of 都市

---

3. causes（因果）

例: 密度 → 渋滞

---

4. similar（類似）

例: 放射型 ≒ 星型

---

5. contrast（対比）

例: 都市 vs 農村

---

6. instance-of（事例）

例: 東京 instance-of 都市

使用ルール

必ず関係を明示する

関連

- 99_oldzettelkasten/04_knowledge_graph/Concept Links
- 99_oldzettelkasten/04_knowledge_graph/Knowledge Graph